

Developer-Setup – Strategy Builder V2

Lokale Vollumgebung mit BSC-Fork · frisch nach `git clone` · Stand: 15.06.2026

Von null bis zur laufenden Full-Stack-Umgebung (Contracts-Fork + Backend + Frontend + DB). Die **Reihenfolge ist wichtig**: erst Fork & Deploy, dann Adressen in die `.env`-Dateien, dann Services starten — sonst laufen Backend/Frontend mit leeren Contract-Adressen.

Goldene Regel: `Fork` → `Deploy` → `.env` füllen → `Seed` → `Services` . Jeder Schritt unten baut darauf auf.

0 · Voraussetzungen

Tool	Version	Prüfen
Node.js	22+	<code>node -v</code>
pnpm	9+	<code>pnpm -v</code> · sonst <code>npm i -g pnpm</code>
Docker	aktiv (für PostgreSQL)	<code>docker ps</code>
MetaMask	Browser-Extension	für das Frontend
Archiv-RPC für BSC	BlastAPI / Alchemy / QuickNode	Pflicht für den Fork (s. u.)

Ports im Überblick

Dienst	Port	Chain-ID
Hardhat-Fork-Node	8545	31337
Backend (NestJS)	3001	—
Frontend (Vite)	5173	—
PostgreSQL	5432 (Test-DB 5433)	—

1 · Dependencies installieren

```
pnpm install
```

Installiert alle Workspace-Pakete (`packages/*`) und baut die nativen Deps (Prisma, esbuild, keytar, NestJS). Generiert den Prisma-Client.

2 · Env-Dateien aus den Vorlagen anlegen

Für jedes Paket existiert eine `.env.example` . Die Contract-Adressen bleiben vorerst leer und werden nach dem Deploy (Schritt 4) eingetragen.

```
cp packages/contracts/.env.example packages/contracts/.env
cp packages/backend/.env.example packages/backend/.env
cp packages/frontend/.env.example packages/frontend/.env
```

Jetzt das Wichtigste in `packages/contracts/.env` setzen: einen archivfähigen BSC-RPC. Öffentliche RPCs (`bsc-dataseed.binance.org`) scheitern beim Forken mit „missing trie node“.

```
# packages/contracts/.env
BSC_MAINNET_RPC_URL=https://bsc-mainnet.public.blastapi.io # oder eigener Alchemy/QuickNode-Key
DEPLOYER_PRIVATE_KEY=0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80 # Hardhat #0, nur lokal!
```

3 · BSC-Fork starten Terminal 1

```
pnpm contracts:fork:bsc
```

Startet einen Hardhat-Node, geforkt von BSC-Mainnet, auf `http://localhost:8545` (Chain-ID 31337). **Terminal offen lassen.**

4 · Contracts deployen Terminal 2

Sobald der Fork läuft, das komplette System deployen (kompiliert automatisch mit *production build profile* — ohne Optimizer schlägt das Proxy-Deployment fehl):

```
pnpm contracts:deploy:fork
```

Deployt FeeRegistry, Vault-Factory + Implementation, MockPriceOracle, die Beispiel-Conditions/Actions **und die 2 DeFi-Registries + 11 DeFi-Actions** (Aave V3 + PancakeSwap V3 inkl. Swap-to-Range-Ratio & Wick-&-Wait-Condition), konfiguriert Fees + Gas-Kompensation und fundet das Test-Wallet. Alle Adressen werden ausgegeben und nach `packages/contracts/deployments/fork-latest.json` geschrieben.

Nur DeFi-Actions nachziehen (Basissystem steht schon)? — `npm run hardhat run --network localhost scripts/deploy-defi-actions.ts` deployt nur die Registries + Actions und *merged* die Adressen in `fork-latest.json` (Factory/Vaults bleiben unberührt). Danach neu seeden.

5 · Adressen in die Service-Env-Dateien übertragen

Aus der Deploy-Ausgabe (bzw. `fork-latest.json`) die Adressen kopieren:

packages/backend/.env

```
NODE_ENV=development           # Pflicht: Balance-Reads über lokales RPC
DATABASE_URL=postgresql://pecunity:pecunity@localhost:5432/pecunity
RPC_URL=http://localhost:8545
FACTORY_ADDRESS=0x...          # StrategyBuilderVaultFactory
FEE_REGISTRY_ADDRESS=0x...      # FeeRegistry
INDEXER_CONFIRMATIONS=0        # Pflicht am Fork (idle fork mined keine Confirmations)
JWT_SECRET=<zufälliger-hex-wert>
FRONTEND_URL=http://localhost:5173
PORT=3001
```

packages/frontend/.env

```
VITE_API_URL=http://localhost:3001
VITE_FACTORY_ADDRESS=0x...      # gleiche Factory-Adresse
```

INDEXER_CONFIRMATIONS=0 nicht vergessen: Sonst erscheinen am Fork frische Deposits/Executions nie in der History (ein idle Fork mined die Confirmation-Blöcke nicht).

6 · Datenbank, Migration & Services starten Terminal 3

```
pnpm dev
```

Ein Befehl erledigt: Backend-`.env` aus Vorlage anlegen (falls fehlt) · PostgreSQL via Docker hochfahren · auf DB warten · `prisma migrate dev` · `shared` bauen · Backend (`:3001`) + Frontend (`:5173`) im Watch-Mode starten.

7 · StepType-Katalog seeden einmalig, nach dem Deploy

Der Seed liest die Adressen aus `fork-latest.json` — deshalb **nach** Deploy & mit laufender DB ausführen (z. B. in einem 4. Terminal, während `pnpm dev` läuft):

```
pnpm db:seed
```

Seedet eine Zeile pro Step (3 Conditions + Beispiel-Actions + DeFi-Actions) + die Token-Listen und überspringt alles, was noch auf `address(0)` steht. **Ohne Seed nach einem Deploy** encoden neue Automations die toten Alt-Adressen und reverten.

Nach jedem erneuten Full-Deploy neu seeden. Bei einem Redeploy bekommen Contracts neue Adressen; alte StepType-Zeilen können sonst als *Duplikate* im Editor hängenbleiben. Tipp: für bestehende Setups lieber inkrementell `deploy-defi-actions.ts` nutzen.

8 · MetaMask verbinden

Eigenes Netzwerk hinzufügen: **RPC** `http://localhost:8545` · **Chain-ID** `31337` · **Currency** ETH. Das Frontend schaltet im Dev-Modus automatisch auf Chain 31337.

Test-Account importieren — das Deploy-Skript fundet `0xBcd4042DE499D14e55001Ccb824a551F3b954096` mit 150 USDT + 1 WBNB:

```
# Hardhat Account #5
0x47e179ec197488593b187f80a00eb0da91f1b9d0b13f8733639f19c30a34926a
```

Dann <http://localhost:5173> öffnen und Wallet verbinden.

Fertig. Du hast jetzt: Fork-Node (T1), Deploy erledigt (T2), DB+Backend+Frontend (T3), Katalog geseedet. Anmelden via SIWE, Vault anlegen, Automations bauen.

9 · Optional – Keeper für Public-Automations

Public-Automations (mit Trigger) werden von einem externen Keeper ausgeführt, sobald der Trigger erfüllt ist. Am Fork manuell anstoßen:

```
pnpm contracts:execute:fork
```

Synct zuerst die Fork-Uhr auf Wall-Clock, führt dann jede aktive, public, getriggerte Automation aus und meldet Fehlschläge (Reverts) ans Backend (wenn `KEEPER_INGEST_SECRET` in beiden `.env` gesetzt ist). Voraussetzung: die Vault hat eine Gas-Reserve (`minFeeDeposit` > 0 + gefüllt), sonst Revert `InsufficientFeeDeposit`.

10 · Optional – MCP-Server (KI-Steuerung)

```
pnpm --filter mcp build
pnpm --filter mcp run init      # Keystore-Passwort im OS-Keychain ablegen
pnpm --filter mcp inspect      # MCP-Inspector gegen den gebauten Server
```

Schreibende Tools sind nur aktiv, wenn `PECUNITY_RPC_URL` (und für `create_vault` auch `PECUNITY_FACTORY_ADDRESS`) gesetzt sind. Details: [packages/mcp/README.md](#).

11 · Optional – Dev-Tooling (OpenSpec / Claude Code)

Nur falls du mit dem Spec-Workflow dieses Repos arbeitest:

```
./scripts/bootstrap.sh
```

Prüft Node, installiert/initialisiert OpenSpec, listet die Projekt-Skills. Superpowers wird einmalig *innerhalb* einer Claude-Session installiert (siehe Ausgabe des Skripts). Danach `WORKFLOW.md` lesen.

12 · Tests

```
pnpm contracts:test      # Hardhat (Solidity)
pnpm backend:test        # Jest (NestJS)
pnpm frontend:test       # Vitest (React)
pnpm --filter shared test
pnpm --filter mcp test
```

13 · Nützliche Befehle

Befehl	Zweck
<code>pnpm dev</code>	DB + Migration + Backend + Frontend
<code>pnpm db:up</code> / <code>db:down</code>	nur PostgreSQL starten/stoppen
<code>pnpm db:migrate</code>	Prisma-Migrationen
<code>pnpm db:seed</code>	StepTypes aus <code>fork-latest.json</code> (neu) seeden
<code>pnpm db:studio</code>	Prisma Studio (DB-GUI)
<code>pnpm contracts:compile</code>	kompilieren + ABIs ins Frontend extrahieren
<code>pnpm contracts:deploy:fork</code>	Full-Deploy auf den Fork

14 · Troubleshooting

Symptom	Ursache / Lösung
Fork-Start: „missing trie node“	Kein Archiv-RPC. <code>BSC_MAINNET_RPC_URL</code> auf BlastAPI/Alchemy/QuickNode setzen.

Deploy bricht mit StackOverflow ab	<i>production build profile</i> nötig — <code>contracts:deploy:fork</code> macht das automatisch; nicht den nackten <code>hardhat run</code> ohne <code>--build-profile production</code> nutzen.
Editor zeigt nur manche / doppelte Step-Nodes	DeFi-Actions nicht deployed bzw. Alt-Adressen in der DB. Deployen + <code>pnpm db:seed</code> (bei Duplikaten Tabelle vorher leeren).
Neue Automation revertet sofort / Keeper „trigger not met“	StepTypes nach dem Deploy nicht neu geseedet → tote Alt-Adressen. Neu seeden <i>und</i> Automation neu deployen.
Deposits/Executions tauchen nie in der History auf	<code>INDEXER_CONFIRMATIONS=0</code> in der Backend- <code>.env</code> setzen, Backend neu starten.
Portfolio/Balances bleiben leer	<code>NODE_ENV=development</code> in der Backend- <code>.env</code> (liest dann über lokales RPC statt Alchemy).
Geänderte <code>.env</code> wirkt nicht	<code>pnpm dev</code> neu starten (Env wird beim Start gelesen).
Docker-Fehler bei <code>pnpm dev</code>	Docker-Daemon starten (<code>docker ps</code> muss laufen).

Quelle: Repo-README, `scripts/dev.mjs`, `docker-compose.yml` und die `.env.example`-Dateien. Private Keys sind die öffentlichen Hardhat-Test-Keys — niemals außerhalb lokaler Forks verwenden.